形式语言与自动机知识点整理

上下文无关文法、下推自动机与上下文无关语言性质

目录

1 上下文无关文法 CFG **2**

1.1 引入动机 2
1.2 回文语言示例 2
1.3 CFG 的形式化定义 2
1.4 符号约定 3
1.5 文法设计例题 3

2 派生、归约与文法语言 **4**

2.1 派生与归约 4
2.2 最左派生与最右派生 4
2.3 文法的语言与上下文无关语言 4
2.4 句型、左句型、右句型和句子 4
2.5 文法等价 5

3 语法分析树与二义性 **5**

3.1 语法分析树 5
3.2 语法树与派生的等价性 5
3.3 文法与语言的二义性 5

4 文法化简与范式 **6**

4.1 化简目标 6
4.2 消除无用符号 6
4.3 消除 epsilon-产生式 6
4.4 消除单元产生式 7
4.5 乔姆斯基范式 CNF 7
4.6 格雷巴赫范式 GNF 7

5 下推自动机 PDA **8**
5.1 PDA 的直观含义 8
5.2 PDA 的形式化定义 8
5.3 PDA 状态转移图 8
5.4 PDA 设计例题 9

6 PDA 的瞬时描述与接受方式 **9**
6.1 瞬时描述 ID 9
6.2 终态接受 10
6.3 空栈接受 10
6.4 两种接受方式的等价性 10

7 CFG 与 PDA 的等价性 **10**
7.1 CFG 到 PDA 10
7.2 GNF 文法到 PDA 11
7.3 PDA 到 CFG 11

8 确定型下推自动机 DPDA **12**
8.1 DPDA 定义 12
8.2 DPDA 与语言类关系 12

9 上下文无关语言的性质 **12**
9.1 CFL 泵引理 12
9.2 泵引理证明思路 13
9.3 用 CFL 泵引理证明非 CFL 的模板 13
9.4 CFL 泵引理例题 13

10 CFL 的封闭性 **14**
10.1 代换封闭性 14
10.2 封闭运算 14
10.3 不封闭运算 15
10.4 与正则语言取交 15

11 CFL 的判定性质 **15**
11.1 可判定问题 15
11.2 不可判定问题 16

12 乔姆斯基文法体系 **16**

1 上下文无关文法 CFG

1.1 引入动机

上下文无关文法（Context-Free Grammar, CFG）具有比正则表达式和有限自动机更强的表达能力，可以描述许多非正则语言，例如

$$\{0^n 1^n \mid n \geq 0\}$$

以及回文语言。CFG 足够简单，可以构造有效的语法分析算法，用于检验给定字符串是否由某个文法产生，因此在程序设计语言设计、编译器实现、XML 的 DTD 等场景中有重要应用。

1.2 回文语言示例

字符串 w 若满足 $w = w^R$，称为回文。若一个语言中的字符串都是回文，则称其为回文语言。

在字母表 $\{0, 1\}$ 上，回文语言可用递归结构定义：

- ε、0、1 都是回文；
- 若 w 是回文，则 $0w0$ 和 $1w1$ 也是回文。

对应 CFG 可写为

$$S \to 0S0 \mid 1S1 \mid 0 \mid 1 \mid \varepsilon.$$

1.3 CFG 的形式化定义

上下文无关文法是四元组

$$G = (V, T, P, S),$$

其中：

- V：变元的有穷集，变元也称为非终结符或语法范畴；
- T：终结符的有穷集，即字母表，且 $V \cap T = \emptyset$；
- P：产生式的有穷集，每个产生式形如 $A \to \alpha$，其中 $A \in V$，$\alpha \in (V \cup T)^*$；
- $S \in V$：初始符号，表示文法开始的地方。

产生式 $A \to \alpha$ 中，A 称为产生式的头或左部，α 称为体或右部。若多个产生式左部相同，可简写为

$$A \to \alpha_1 \mid \alpha_2 \mid \cdots \mid \alpha_k.$$

1.4 符号约定

常见约定如下：

- 大写字母 $A, B, C, \ldots$ 表示变元；
- 小写字母 $a, b, c, \ldots$ 表示终结符；
- X, Y, Z 表示终结符或非终结符；
- u, v, w 表示终结符串；
- α, β, γ 表示终结符或非终结符组成的串。

1.5 文法设计例题

例题 1：回文语言（课堂例题）

$$L = \{w \in \{0,1\}^* \mid w = w^R\}.$$

可用文法

$$S \to 0S0 \mid 1S1 \mid 0 \mid 1 \mid \varepsilon.$$

例如 0110 的一个派生为

$$S \Rightarrow 0S0 \Rightarrow 01S10 \Rightarrow 0110.$$

例题 2：$\{0^n 1^m \mid n \neq m\}$（课堂例题） 语言可分成 $n > m$ 和 $n < m$ 两种情况：

$$S \to AC \mid CB, \qquad C \to 0C1 \mid \varepsilon,$$

$$A \to A0 \mid 0, \qquad B \to 1B \mid 1.$$

其中 C 生成数量相等的 0 和 1，A 补充至少一个额外的 0，B 补充至少一个额外的 1。

例题 3：$\{a^n b^n c^m \mid n \geq 1, m \geq 0\}$（2026 年期末考） 令

$$G = (\{S, A, B\}, \{a, b, c\}, P, S),$$

其中

$$S \to AB, \qquad A \to aAb \mid ab, \qquad B \to cB \mid \varepsilon.$$

A 生成至少一对匹配的 $a^n b^n$，B 生成任意个 c。

例题 4：$\{a^m b^{2n} a^m \mid m \geq 1, n \geq 1\}$（2026 年期末考） 文法可写为

$$S \to aSa \mid aAa, \qquad A \to bbA \mid bb.$$

S 生成首尾数量相同且至少一对的 a，A 生成至少两个且数量为偶数的 b。

例题 5：$\{a^n b^m \mid n \geq m \geq 0\}$（**2024 年期末考**） 一种文法为

$$S \to aSb \mid A, \qquad A \to aA \mid \varepsilon.$$

前一类产生式生成成对的 a, b，后一类产生式补充额外的 a。

2 派生、归约与文法语言

2.1 派生与归约

从字符串到文法变元的分析过程称为归约，自底向上，由产生式的体向头分析；从文法变元到字符串的分析过程称为派生，自顶向下，由产生式的头向体分析。

若存在产生式 $A \to \beta$，且 $\alpha, \gamma \in (V \cup T)^*$，则

$$\alpha A \gamma \Rightarrow \alpha \beta \gamma.$$

这表示用产生式右部 β 替换串中的变元 A。若经过零步或多步派生，记为 $\Rightarrow^*$；若刚好经过 n 步，可记为 $\Rightarrow^n$。

2.2 最左派生与最右派生

只替换符号串中最左边变元的派生称为最左派生，记为 $\Rightarrow_{\text{lm}}$；只替换最右边变元的派生称为最右派生，记为 $\Rightarrow_{\text{rm}}$。对同一棵完整语法分析树（等价地，对 $S \Rightarrow^* w$ 且 w 为终结串的完整派生），存在相应的最左派生和最右派生；但并非任意以含变元的句型为终点的部分派生都能与某个最左派生或最右派生一一对应。

2.3 文法的语言与上下文无关语言

文法 $G = (V, T, P, S)$ 的语言定义为

$$L(G) = \{w \in T^* \mid S \Rightarrow^* w\}.$$

也就是说，w 必须仅由终结符组成，并且初始符号 S 能派生出 w。

若语言 L 是某个 CFG 定义的语言，即存在 G 使得 $L = L(G)$，则称 L 为上下文无关语言（Context-Free Language, CFL）。所谓上下文无关，是指每一步中某个变元只根据它自己的产生式替换，不依赖它左右两侧的上下文。

2.4 句型、左句型、右句型和句子

若 $S \Rightarrow^* \alpha$，则 α 称为文法的句型。若 $S \Rightarrow_{\text{lm}}^* \alpha$，则 α 称为左句型；若 $S \Rightarrow_{\text{rm}}^* \alpha$，则 α 称为右句型。只含终结符的句型称为句子，文法的语言就是它全部句子的集合。

2.5 文法等价

若两个文法 G_1 和 G_2 满足

$$L(G_1) = L(G_2),$$

则称 G_1 和 G_2 等价。

3 语法分析树与二义性

3.1 语法分析树

CFG 的语法分析树（parse tree）也称派生树。其定义要点：

- 每个内节点标记为变元；
- 每个叶节点标记为 $V \cup T \cup \{\varepsilon\}$ 中的符号；
- 若某内节点标记为 A，子节点从左到右标记为 $X_1, \ldots, X_k$，则 $A \to X_1 \cdots X_k$ 是产生式；
- 若某个子节点标记为 ε，则它必须是唯一子节点，并且存在产生式 $A \to \varepsilon$。

语法分析树的全部叶节点从左到右连接，称为该树的产物或结果。若根节点为开始符号 S，叶节点全为终结符或 ε，则产物属于 $L(G)$。

3.2 语法树与派生的等价性

对 CFG G，有

$$A \Rightarrow^* w$$

当且仅当存在以 A 为根、产物为 w 的语法分析树。

每棵语法分析树都有唯一的最左派生和唯一的最右派生。对同一个串，不同的普通派生顺序可能对应同一棵语法树，但不同的最左派生一定对应不同的语法树。

3.3 文法与语言的二义性

如果某个 CFG 使某个终结串 $w \in L(G)$ 有两棵不同的完整语法分析树（等价地，有两个不同的最左派生或两个不同的最右派生），则称该文法是二义的。

有些文法的二义性可以通过重新设计文法消除。例如算术表达式中若不区分运算优先级和结合性，表达式可能有多棵语法树；通过引入表达式、项、因子等层级，可以消除这类二义性。

如果一个上下文无关语言的所有文法都是二义的，则称该语言是固有二义的。固有二义语言确实存在。

4 文法化简与范式

4.1 化简目标

文法形式非常自由，过于复杂不利于自动处理。常见化简目标包括：

- 消除无用符号；
- 消除 ε-产生式；
- 消除单元产生式；
- 转换为乔姆斯基范式或格雷巴赫范式。

4.2 消除无用符号

对 CFG 中的符号 X：

- 若 $S \Rightarrow^* \alpha X \beta$，称 X 是可达的；
- 若 $X \Rightarrow^* w$ 且 $w \in T^*$，称 X 是可产生的；
- 若 X 同时可达且可产生，称 X 是有用的；否则称为无用符号。

计算方法：

- 产生性：终结符都是可产生的；若 $A \to \alpha$ 且 α 中符号都可产生，则 A 可产生；
- 可达性：S 可达；若 A 可达且 $A \to \alpha$，则 α 中的符号可达；
- 删除含非产生符号或非可达符号的产生式。

可靠顺序是先消除非产生符号，再消除非可达符号。每个非空 CFL 都能由不带无用符号的 CFG 定义。

4.3 消除 epsilon-产生式

形如 $A \to \varepsilon$ 的产生式称为 ε-产生式。若 $A \Rightarrow^* \varepsilon$，称 A 是可空的。

确定可空变元：

- 若 $A \to \varepsilon$，则 A 可空；
- 若 $A \to X_1 \cdots X_k$ 且每个 X_i 都可空，则 A 可空。

替换产生式时，若某产生式右部含有可空变元，则对每个可空变元考虑“保留”或“删除”两种情况，但不能把右部删成空串，除非专门处理开始符号产生空串的情形。

若语言不含 ε，可消除全部 ε-产生式并保持语言不变；若语言含 ε，通常新增一个不在任何产生式右部出现的新开始符号 S'，并加入 $S' \to S \mid \varepsilon$。这样 S' 专门负责产生 ε，而 S 继续产生语言中的全部非空串；若只让新开始符号产生 ε 而不保留 $S' \to S$，会丢失非空串。

4.4 消除单元产生式

形如 $A \to B$ 且 $A, B \in V$ 的产生式称为单元产生式。

若 $A \Rightarrow^* B$ 且派生过程中只使用单元产生式，则称 (A, B) 为单元对。消除方法：

1. 找出全部单元对；
2. 删除全部单元产生式；
3. 对每个单元对 (A, B)，把 B 的非单元产生式复制给 A。

可靠的完整化简顺序为：

消除ε-产生式 → 消除单元产生式 → 消除非产生符号 → 消除非可达符号.

4.5 乔姆斯基范式 CNF

乔姆斯基范式（Chomsky Normal Form, CNF）要求每个产生式形如

$$A \to BC \quad 或 \quad A \to a,$$

其中 A, B, C 是变元，a 是终结符。对于不含 ε 的 CFL，都存在等价 CFG 满足 CNF。

CFG 转 CNF 的基本方法：

1. 先化简文法，消除无用符号、ε-产生式和单元产生式；
2. 对长度大于 1 的右部中的终结符 a，引入新变元 $C_a \to a$ 替换；
3. 对右部长度大于 2 的产生式，引入新变元，将长串拆成二元级联。

若语言不含 ε，CNF 中长度为 n 的非空串恰好需要 $2n-1$ 步派生。

4.6 格雷巴赫范式 GNF

格雷巴赫范式（Greibach Normal Form, GNF）要求每个产生式形如

$$A \to a\alpha,$$

其中 A 是变元，a 是终结符，α 是零个或多个变元组成的串。对于不含 ε 的 CFL，都存在等价 CFG 满足 GNF。

GNF 的特点是每个产生式都会引入一个终结符；在不含 ε 的情形下，长度为 n 的串恰好需要 n 步派生。

5 下推自动机 PDA

5.1 PDA 的直观含义

下推自动机（Pushdown Automaton, PDA）可以看作 ε-NFA 加上一个栈。它的动作由三个因素决定：

- 当前状态；
- 当前输入符号，或 ε；
- 当前栈顶符号。

栈具有后进先出特性，只能访问栈顶，长度无限，可用于存储历史信息。PDA 具备与 CFG 等价的定义语言能力。

5.2 PDA 的形式化定义

PDA 是七元组

$$P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F),$$

其中：

- Q：有穷状态集；
- Σ：有穷输入符号集；
- Γ：有穷栈符号集；
- $\delta : Q \times (\Sigma \cup \{\varepsilon\}) \times \Gamma \to \mathcal{P}(Q \times \Gamma^*)$：状态转移函数；
- q_0：初始状态；
- Z_0：初始栈底符号；
- F：接受状态集。

若

$$(p, \gamma) \in \delta(q, a, X),$$

表示 PDA 在状态 q，读入 a（可为 ε），栈顶为 X 时，可以转到状态 p，并用 γ 替换栈顶 X。若 $\gamma = \varepsilon$，表示弹栈；若 $\gamma = X$，栈顶不变；若 $\gamma = Y_1 \cdots Y_k$，则用该串替换 X，其中最左符号成为新的栈顶。

5.3 PDA 状态转移图

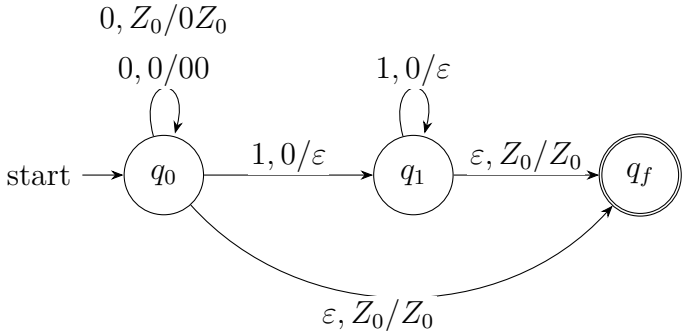
PDA 状态图中的边标签常写为

$$a,\ X/\gamma,$$

含义是：消耗输入 a（或 ε），弹出栈顶 X，压入 γ。

5.4 PDA 设计例题

例题 1：识别 $\{0^n1^n \mid n \geq 0\}$（课堂例题） 思路：读入 0 时压栈；读入 1 时每次弹出一个 0；输入结束且栈底 Z_0 露出时进入接受状态。



该 PDA 以终态方式接受；从 q_0 到 q_f 的空转移负责接受 $n = 0$ 的空串。

例题 2：识别偶长度回文语言 $\{ww^R \mid w \in \{0,1\}^*\}$（课堂例题） 注意：$\{ww^R\}$ 只包含偶长度回文（w 与其反转 w^R 直接连接），不识别单个中间字符的奇长度回文。思路：先非确定地把前半部分压栈；在某个时刻通过 ε-转移猜测中点；随后输入符号必须与栈顶匹配并弹栈；栈底露出时接受。

例题 3：由 CFG 构造 PDA（2026 年期末考） 对文法

$$S \to AB, \qquad A \to aAb \mid ab, \qquad B \to cB \mid \varepsilon,$$

构造以空栈方式接受的 PDA。按 CFG 到 PDA 的通用构造，使用单状态 q，初始栈符号为 S：

$$P = (\{q\}, \{a,b,c\}, \{a,b,c,S,A,B\}, \delta, q, S, \emptyset).$$

转移包括：

$$\begin{aligned}
\delta(q,\varepsilon,S) &= \{(q,AB)\},\\
\delta(q,\varepsilon,A) &= \{(q,aAb),(q,ab)\},\\
\delta(q,\varepsilon,B) &= \{(q,cB),(q,\varepsilon)\},\\
\delta(q,a,a) &= \{(q,\varepsilon)\},\\
\delta(q,b,b) &= \{(q,\varepsilon)\},\\
\delta(q,c,c) &= \{(q,\varepsilon)\}.
\end{aligned}$$

6 PDA 的瞬时描述与接受方式

6.1 瞬时描述 ID

PDA 某一瞬间的格局称为瞬时描述（Instantaneous Description, ID），写作

$$(q, w, \alpha),$$

其中 q 是当前状态，w 是剩余输入串，α 是栈内容，且栈顶符号写在最左边。

若 $(p,\gamma)\in\delta(q,a,X)$，则

$$(q,aw,X\beta)\vdash(p,w,\gamma\beta),$$

其中 a 可以是 ε。记 $\vdash^*$ 表示经过零步或多步转移。

6.2 终态接受

PDA 以终态方式接受的语言记为 $L(P)$：

$$L(P)=\{w\in\Sigma^*\mid(q_0,w,Z_0)\vdash^*(q,\varepsilon,\alpha),\ q\in F\}.$$

即输入被消耗完，并且到达某个接受状态；栈中剩余内容不影响终态接受。

6.3 空栈接受

PDA 以空栈方式接受的语言记为 $N(P)$：

$$N(P)=\{w\in\Sigma^*\mid(q_0,w,Z_0)\vdash^*(q,\varepsilon,\varepsilon)\}.$$

即输入被消耗完，且栈被清空；是否到达终态不重要。

6.4 两种接受方式的等价性

PDA 的终态接受和空栈接受在语言表达能力上等价：

- 若某语言可由 PDA 以终态方式接受，则存在另一个 PDA 以空栈方式接受同一语言；
- 若某语言可由 PDA 以空栈方式接受，则存在另一个 PDA 以终态方式接受同一语言。

构造思想都是先模拟原 PDA，再通过新增开始状态、接受状态或清栈状态，并借助特殊栈底符号，避免误清空栈或误进入终态。

7 CFG 与 PDA 的等价性

7.1 CFG 到 PDA

由 CFG 构造 PDA 的核心思想是：PDA 模拟文法的最左派生。

给定 CFG $G=(V,T,P,S)$，可构造以空栈方式接受的 PDA

$$P=(\{q\},T,V\cup T,\delta,q,S,\emptyset).$$

转移规则：

- 对每个产生式 $A \to \alpha$，加入
$$\delta(q, \varepsilon, A) \ni (q, \alpha);$$
- 对每个终结符 $a \in T$，加入
$$\delta(q, a, a) \ni (q, \varepsilon).$$

含义：若栈顶是变元，就非确定地用某个产生式右部替换它；若栈顶是终结符，必须与输入符号匹配并弹出。

7.2 GNF 文法到 PDA

若 CFG 已是 GNF，每个产生式形如

$$A \to a\alpha,$$

则 PDA 可以在应用产生式时同时读入终结符 a：

$$\delta(q, a, A) \ni (q, \alpha).$$

这比一般 CFG 到 PDA 的构造更直接。

7.3 PDA 到 CFG

若 PDA 以空栈方式接受语言，则可构造等价 CFG。核心变元形如

$$[qXp],$$

含义是：从状态 q 出发，处理并弹出栈顶符号 X，最终到达状态 p，过程中消耗的输入串就是该变元能派生出的串。

对 PDA 转移

$$(r, Y_1Y_2\cdots Y_k) \in \delta(q, a, X),$$

构造产生式的大意为

$$[qXp] \to a[rY_1r_1][r_1Y_2r_2]\cdots[r_{k-1}Y_kp],$$

其中中间状态 $r_1, \ldots, r_{k-1}$ 遍历所有可能状态。若 $k = 0$，即弹出 X，则产生式为

$$[qXp] \to a$$

并要求 $p = r$。开始符号通过所有可能终止状态连接到 $[q_0Z_0p]$，即加入

$$S \to [q_0Z_0p] \qquad (p \in Q).$$

该构造说明：PDA 能接受的语言都是 CFL；结合 CFG 到 PDA，可得 CFL 与 PDA 接受语言等价。

8 确定型下推自动机 DPDA

8.1 DPDA 定义

若 PDA 满足：

- 对任意状态、输入符号或 ε、栈顶符号，至多有一个可选动作；
- 若某个状态和栈顶符号下存在 ε-动作，则同一状态和栈顶符号下不能再存在读入实际输入符号的动作，

则称其为确定型下推自动机（Deterministic Pushdown Automaton, DPDA）。DPDA 在任何情况下都不需要选择可能的移动。

DPDA 与一般 PDA 的识别能力不等价。DPDA 以终态方式接受的语言称为确定的上下文无关语言（DCFL）。

8.2 DPDA 与语言类关系

- 每个正则语言都可由某个 DPDA 以终态方式接受，因为 DPDA 可以不用栈而模拟 DFA；
- DPDA 接受语言是真正受限的一类 CFL；
- 若语言可由 DPDA 接受，则存在无二义 CFG 定义该语言；
- 并非所有无二义 CFL 都能由 DPDA 识别。

空栈方式的 DPDA 还受到前缀性质影响。语言 L 满足前缀性质，是指不存在两个不同串 $x, y \in L$ 使 x 是 y 的真前缀。一个语言可由某个 DPDA 以空栈方式接受，当且仅当它具有前缀性质，并且可由某个 DPDA 以终态方式接受。

9 上下文无关语言的性质

9.1 CFL 泵引理

若语言 L 是 CFL，则存在正整数 N，对任意 $w \in L$，只要 $|w| \geq N$，就可以将 w 分为五部分

$$w = uvxyz$$

满足：

$$|vxy| \leq N, \qquad |vy| > 0, \qquad uv^i xy^i z \in L \quad (i \geq 0).$$

也就是说，v 和 y 可以同步泵入或泵出，所得字符串仍属于该语言。

9.2 泵引理证明思路

证明使用 CNF 文法。若 CFG 有 m 个变元，取 $N = 2^m$。在 CNF 派生树中，若产物长度至少 N，则某条根到叶路径经过至少 $m+1$ 个变元节点，其中必有两个标记相同的变元（记为 A）。选择离叶节点最近的一对相同变元 A：从上层 A 到叶的路径上所有变元互不相同，路径高度至多 m，对应的子树产物长度至多 $2^m = N$，这就给出了约束 $|vxy| \le N$。同时，上层 A 派生到下层 A 至少经过一步产生式，且 CNF（除 $S \to \varepsilon$ 外）没有 ε-产生式，因此 v 与 y 不会同时为空，即 $|vy| > 0$。利用这两个相同变元之间的子树重复，可得到 uv^ixy^iz 仍属于语言。

9.3 用 CFL 泵引理证明非 CFL 的模板

1. 假设 L 是 CFL；
2. 取泵引理常数 N；
3. 选择一个结构刚性的长串 $w \in L$；
4. 对任意分解 $w = uvxyz$，利用 $|vxy| \le N$ 分析 vxy 最多跨越哪些区段；
5. 选取 $i = 0$ 或 $i = 2$，破坏语言要求；
6. 与泵引理矛盾，故 L 不是 CFL。

9.4 CFL 泵引理例题

例题 1：$\{0^n1^n2^n \mid n \ge 1\}$ 不是 CFL（课堂例题） 假设该语言是 CFL，取泵引理常数 N，令

$$w = 0^N1^N2^N.$$

由于 $|vxy| \le N$，子串 vxy 不可能同时覆盖 0 区、1 区、2 区，只可能完全在某一区，或跨越相邻两区。泵入或泵出 v, y 后，至少有一个区段的数量改变，而不可能同时保持三段数量相等，矛盾。因此该语言不是 CFL。

例题 2：$\{0^i1^j2^k \mid i > j > k \ge 1\}$ 不是 CFL（2026 年期末考） 假设 L 是 CFL，取泵引理常数 N，令

$$w = 0^{N+2}1^{N+1}2^N.$$

该串满足 $i > j > k$。由于 $|vxy| \le N$，vxy 不可能同时覆盖三段，只能位于某一段或跨越相邻两段。若位于单段，泵入或泵出会改变单段数量，从而破坏某个严格不等式；若跨越 0/1 或 1/2 边界，泵入或泵出会改变相邻两段的数量或破坏 $0^*1^*2^*$ 的顺序。总能取 $i = 0$ 或 $i = 2$ 使所得串不属于 L，矛盾。

例题 3：$\{a^k b a^k b a^k b \mid k > 0\}$ **不是 CFL（2024 年期末考）** 假设 L 是 CFL，取泵引理常数 N，令

$$w = a^N b a^N b a^N b.$$

由于 $|vxy| \le N$，vxy 最多落在某一个 a 块内部，或跨越一个 b 附近，不能同时覆盖三个 a 块。泵入或泵出只会改变至多两个局部区域，无法保持三个 a 块长度同时相等；若 v 或 y 含有 b，泵入还会改变分隔符数量或顺序。因此存在某个泵值使串不再属于 L，矛盾。

10 CFL 的封闭性

10.1 代换封闭性

代换是从字母表 Σ 到语言集合的函数，即每个字符 $a \in \Sigma$ 被映射为某个语言 $s(a)$。扩展到字符串：

$$s(a_1 a_2 \cdots a_n) = s(a_1)s(a_2)\cdots s(a_n),$$

扩展到语言：

$$s(L) = \bigcup_{w \in L} s(w).$$

若 L 是 CFL，且每个 $s(a)$ 都是 CFL，则 $s(L)$ 也是 CFL。

10.2 封闭运算

CFL 对以下运算封闭：

- 并；
- 连接；
- Kleene 闭包；
- 正闭包；
- 代换；
- 同态；
- 逆同态；
- 反转。

并、连接、闭包可以直接用文法构造证明。构造前应先把两个原文法的变元集合（连同各自的开始符号）改名为互不相交的集合，并引入一个全新的开始符号 S（不在任一原文法中出现）。例如若 L_1, L_2 的开始符号分别为 S_1, S_2，则：

$$L_1 \cup L_2 : \quad S \to S_1 \mid S_2,$$

$$L_1L_2: \quad S \to S_1S_2,$$

$$L_1^*: \quad S \to S_1S \mid \varepsilon.$$

10.3 不封闭运算

CFL 对交和补不封闭。典型反例：

$$L_1 = \{0^i1^i2^j \mid i,j \geq 0\}, \qquad L_2 = \{0^i1^j2^j \mid i,j \geq 0\}.$$

L_1 和 L_2 都是 CFL，但

$$L_1 \cap L_2 = \{0^n1^n2^n \mid n \geq 0\}$$

不是 CFL，因此 CFL 对交不封闭。

若 CFL 对补封闭，则由德摩根律

$$L_1 \cap L_2 = \overline{\overline{L_1} \cup \overline{L_2}}$$

可推出 CFL 对交封闭，与上面矛盾，因此 CFL 对补也不封闭。

10.4 与正则语言取交

若 L 是 CFL，R 是正则语言，则

$$L \cap R$$

仍是 CFL。证明思路是用 PDA 识别 L，DFA 识别 R，构造一个积式 PDA 同时模拟二者。

这个性质常用于证明某语言不是 CFL：假设目标语言是 CFL，再与某个正则语言取交，得到已知非 CFL，矛盾。

11 CFL 的判定性质

11.1 可判定问题

CFL 的以下问题可判定：

- 空性：给定 CFG，判断 $L(G)$ 是否为空。只需判断开始符号是否可产生终结符串；
- 成员性：给定 CFG 和串 w，判断 $w \in L(G)$。可先转为 CNF，再用 CYK 算法判断。

11.2 不可判定问题

以下问题不可判定：

- 判断 CFG 是否二义；
- 判断 CFL 是否固有二义；
- 两个 CFL 的交是否为空；
- 两个 CFL 是否相同；
- CFL 的补是否为空；
- CFL 是否等于 Σ^*。

12 乔姆斯基文法体系

若文法产生式都形如 $\alpha \to \beta$，其中 α, β 是符号串，且 α 中至少含一个变元，则：

- 0 型文法：也称短语结构文法；
- 1 型文法：若产生式满足 $|\alpha| \le |\beta|$（长度不减），称为上下文有关文法；标准特例允许 $S \to \varepsilon$，但此时 S 不得出现在任何产生式右部，以保证除空串外的句子仍满足长度不减；
- 2 型文法：若左部是单个变元，即 $A \to \beta$，称为上下文无关文法；
- 3 型文法：若产生式形如 $A \to aB$ 或 $A \to a$，称为正则文法（右线性文法；对称地也可用左线性 $A \to Ba$、$A \to a$，但同一文法中两种形式不能混用）。右线性文法本身不能直接生成 ε；若语言含空串，需允许 $S \to \varepsilon$ 作为特例（S 不出现在任何产生式右部）。

四类文法的表达能力从 3 型到 0 型逐级增强。